

Velocity-Spectrum and VDOS Plotting Specification

VP1: Result-Aware Spectral Plotting

mdstats

2026-07-15

Contents

Purpose and implementation stage	1
Motivation	2
Provenance	2
Borrowed software machinery	2
Reused mdstats scientific machinery	2
mdstats-specific contribution	2
Dependency boundary	2
Public API	3
Input result contract	3
Velocity spectrum	3
VDOS	3
Type safety	3
Horizontal-axis contract	4
Density-coordinate warning	4
Projection contract	4
Total	4
Cartesian components	4
Per atom	4
Vertical-axis labels	5
Velocity spectrum	5
VDOS	5
Explicit display normalization	5
Figure and axes ownership	5
Algorithm	6
Edge cases and failure policy	7
Test specification	7
Usage examples	7
Total VDOS in THz	7
Cartesian velocity spectrum in inverse centimeters	8
Selected per-atom comparison	8
Deferred work	8
References	8

Purpose and implementation stage

This document specifies stage VP1 in

`mdstats/plotting/velocity_spectrum.py`

The public function is

```
plot_velocity_spectrum(result, ...)
```

It plots an already computed `VelocitySpectrumResult` or `VDOSResult`. It does not recompute velocities, a VACF, a Fourier transform, a VDOS normalization, or phonon eigenmodes.

VP1 is implemented in `mdstats 0.19.3a0`.

Motivation

The spectral branch now produces two scientifically distinct result types:

1. a one-sided velocity spectral density;
2. an explicitly normalized finite-temperature VDOS.

A shared plotting function is useful because both results carry the same three stored horizontal coordinates and the same total, Cartesian, and optional per-atom decomposition. The plotter must nevertheless preserve the distinction between the two result types and their units.

The design goals are:

- one result-aware plotting entry point;
- no hidden scientific renormalization;
- no Fourier transformation or interpolation in the plotting layer;
- deterministic component and atom labels;
- bounded implicit per-atom output;
- explicit terminology that does not call a generic VDOS a phonon DOS;
- normal Matplotlib ownership of figures, axes, display, and file output.

Provenance

Borrowed software machinery

Rendering uses Matplotlib. The software reference is Hunter [1]. VP1 relies on Matplotlib line artists, axes labels, legends, and figure ownership; it does not implement an external numerical or mathematical algorithm.

Reused `mdstats` scientific machinery

The input arrays and their physical meanings are owned by:

- `VelocitySpectrumResult` and `compute_vacf_spectrum()` in `VS1`;
- `VDOSResult` and `compute_vdos()` in `VS3`;
- `_spectral_units.py` for the stored THz, inverse-centimeter, and meV axes.

VP1 does not repeat the Wiener-Khinchin transform, one-sided spectral scaling, discrete FFT-bin integration, or VDOS normalization. Their sources and `mdstats` adaptations are documented in the corresponding analysis specifications.

`mdstats`-specific contribution

The following are VP1 design decisions:

- one function accepting either supported result type;
- result-aware vertical labels and terminology;
- a common maximum-absolute display scale across all selected curves;
- a twelve-curve limit for implicit per-atom plotting;
- exact requested atom ordering and strict missing-index rejection;
- the rule that alternate horizontal coordinates do not silently transform the stored ordinate density;
- returning `(Figure, Axes)` for consistency with the existing `mdstats` plotting API.

Dependency boundary

```
VelocitySpectrumResult -----+
                                +--> plot_velocity_spectrum --> Matplotlib artists
VDOSResult -----+
                                +--> plot_vdos_spectrum --> Matplotlib artists
```

The module may import only:

- NumPy for non-mutating array selection and display scaling;
- Matplotlib for plotting;

- the two spectral result types.

It must not import trajectory collections, VACF estimators, FFT helpers, quadrature helpers, or structure-analysis modules.

Public API

```
from collections.abc import Sequence
from typing import Literal

from matplotlib.axes import Axes
from matplotlib.figure import Figure

def plot_velocity_spectrum(
    result: VelocitySpectrumResult | VDOSResult,
    *,
    x_axis: Literal["thz", "cm^-1", "mev"] = "thz",
    projection: Literal["total", "components", "per_atom"] = "total",
    atom_indices: Sequence[int] | None = None,
    normalize_for_display: bool = False,
    ax: Axes | None = None,
) -> tuple[Figure, Axes]:
    ...
```

The original roadmap draft stated an Axes-only return. The implemented contract follows the established `plot_msd()` and `plot_pair_rdf()` convention and returns both the owning figure and axes.

Input result contract

Velocity spectrum

For `VelocitySpectrumResult`, VP1 consumes:

```
frequencies_thz
wavenumbers_cm_inv
energies_mev
scalar_spectrum
component_spectra
per_atom_scalar
per_atom_indices
spectrum_units
```

The constructor invariants already guarantee consistent shapes, finite values, and Cartesian trace identities.

VDOS

For `VDOSResult`, VP1 consumes:

```
frequencies_thz
wavenumbers_cm_inv
energies_mev
total
components
per_atom
per_atom_indices
density_units
```

The constructor invariants already guarantee nonnegative values, consistent frequency axes, and Cartesian trace identities.

Type safety

Any other result type is rejected with `TypeError`. The plotting layer does not attempt duck typing because an arbitrary object may not carry the normalization and terminology guarantees required for a correct vertical label.

Horizontal-axis contract

The accepted axes are:

x_axis	Stored array	Label
"thz"	frequencies_thz	Frequency (THz)
"cm^-1"	wavenumbers_cm_inv	Wavenumber (cm^-1)
"mev"	energies_mev	Energy (meV)

No interpolation or recomputation occurs.

Density-coordinate warning

The ordinate arrays are stored as densities with respect to the canonical THz frequency grid. Selecting inverse centimeters or meV changes only the displayed horizontal coordinate. VP1 does **not** apply a Jacobian such as

$$P_{\tilde{\nu}}(\tilde{\nu}) = P_f(f) \left| \frac{df}{d\tilde{\nu}} \right|.$$

Therefore, the scientific area identity remains defined on the THz grid. This choice follows the roadmap requirement to switch stored coordinates without recomputing data and avoids silently changing the result object in a plotter. A future API may provide explicit density-coordinate conversion as a separate scientific transformation.

Projection contract

Total

projection="total"

plots one curve:

- scalar_spectrum for a velocity spectrum;
- total for a VDOS.

atom_indices is invalid in this mode.

Cartesian components

projection="components"

plots the three stored diagonal projections in the fixed order

x, y, z

and creates a legend. The plotter does not reconstruct the tensor or plot complex off-diagonal cross spectra.

atom_indices is invalid in this mode.

Per atom

projection="per_atom"

plots scalar per-atom curves:

- per_atom_scalar for a velocity spectrum;
- per_atom for a VDOS.

The source result must contain both the per-atom array and canonical per_atom_indices.

Explicit selection

When atom_indices is supplied:

- it must be a nonempty one-dimensional integer sequence;
- duplicate indices are rejected;
- every requested canonical index must be present;

- curves follow the exact user-specified order;
- no missing index is silently omitted.

Implicit selection guard

When `atom_indices` is `None`, all stored per-atom curves may be plotted only when

$$N_{\text{curves}} \leq 12.$$

Larger results require an explicit subset. This is a readability and browser safety guard, not a scientific limit.

Vertical-axis labels

The vertical label is derived from the result type and stored units.

Velocity spectrum

Examples are:

Velocity spectral density ($\text{\AA}^2/\text{ps}$)

Velocity spectral density ($\text{amu } \text{\AA}^2/\text{ps}$)

The plotter does not infer weighting from units; it uses the authoritative `spectrum_units` field.

VDOS

Examples are:

VDOS (THz^{-1})

VDOS (degrees of freedom / THz)

The term phonon DOS is never introduced by VP1. Whether a VDOS has a harmonic phonon interpretation depends on the physical system and belongs to scientific interpretation outside the plotter.

Explicit display normalization

When

`normalize_for_display=True`

VP1 collects the selected curve matrix

$$Y \in \mathbb{R}^{F \times C}$$

and computes one common scale

$$s = \max_{m,c} |Y_{mc}|.$$

The displayed curves are

$$\hat{Y}_{mc} = Y_{mc}/s.$$

A common scale preserves relative amplitudes among all selected curves. It is not a unit-area normalization and does not separately peak-normalize each component or atom.

The operation is performed on a copy. The immutable result arrays and their normalization metadata remain unchanged. When every selected value is zero, VP1 raises `ValueError` rather than dividing by zero.

The vertical label becomes

Display-normalized intensity (arb. units)

Without this explicit flag, the plotted ordinates equal the stored arrays exactly.

Figure and axes ownership

When `ax` is `None`, VP1 creates a new figure and axes with `plt.subplots()`. When an axes is supplied, it is reused and its owning figure is returned.

The function:

- does not call `plt.show()`;
- does not save a file;
- does not close a figure;
- adds a light major grid;
- creates a legend for component and per-atom projections;
- leaves later title, limits, layout, and file-format choices to the caller.

Algorithm

```
validate result type
```

```
validate normalize_for_display and ax
```

```
select stored horizontal coordinate and label
```

```
if projection == total:
```

```
    select one scalar curve
```

```
elif projection == components:
```

```
    select x/y/z matrix
```

```
elif projection == per_atom:
```

```
    validate per-atom availability
```

```
    resolve canonical atom indices to stored columns
```

```
    enforce implicit curve-count guard
```

```
else:
```

```
    reject
```

```
copy selected curve matrix
```

```
if normalize_for_display:
```

```
    scale = maximum absolute selected value
```

```
    reject zero/nonfinite scale
```

```
    divide copied matrix by common scale
```

```
create or reuse Matplotlib axes
```

```
plot each selected column
```

```
set result-aware axis labels
```

```
add grid and projection-dependent legend
```

```
return figure, axes
```

For F frequency bins and C selected curves, time and temporary storage are

$$O(FC).$$

No operation scales with the original trajectory length.

Edge cases and failure policy

Case	Behavior
unsupported result object	TypeError
invalid horizontal axis	ValueError
invalid projection	ValueError
atom_indices outside per-atom mode	ValueError
missing per-atom arrays	ValueError
more than 12 implicit atom curves	ValueError
empty, duplicate, noninteger, or missing atom request	rejected
non-boolean display-normalization flag	TypeError
invalid axes object	TypeError
all-zero selected data with display normalization	ValueError
negative raw velocity-spectrum samples	plotted as stored
VDOS input	plotted nonnegative under its constructor invariants

Negative velocity-spectrum lobes are not clipped by the plotter. Their handling belongs to the upstream spectral estimator or VDOS conversion policy.

Test specification

The focused test module is

tests/test_plot_velocity_spectrum.py

Required coverage:

1. THz, inverse-centimeter, and meV x axes use the stored arrays exactly.
2. Raw and per-weight velocity spectra receive correct unit labels.
3. VDOS results are labeled as VDOS, not phonon DOS.
4. Total curves are unchanged when display normalization is disabled.
5. Cartesian projection produces exactly three trace curves and a legend.
6. Explicit per-atom selection preserves request order.
7. Small per-atom results may be selected implicitly.
8. Large per-atom results require an explicit subset.
9. Missing per-atom data and invalid atom requests are rejected.
10. Display normalization uses one common scale and does not mutate the result.
11. Existing axes are reused.
12. Invalid result types, axes, projections, and options are rejected.
13. Top-level and `mdstats.plotting` imports are available.

The complete package regression suite must pass after the new export is added.

Usage examples

Total VDOS in THz

```
from mdstats import plot_velocity_spectrum

fig, ax = plot_velocity_spectrum(vdos)
fig.tight_layout()
fig.savefig("na_vdos.pdf")
```

Cartesian velocity spectrum in inverse centimeters

```
fig, ax = plot_velocity_spectrum(  
    spectrum,  
    x_axis="cm-1",  
    projection="components",  
)
```

The y density remains in the units recorded by spectrum; only the horizontal coordinate changes.

Selected per-atom comparison

```
fig, ax = plot_velocity_spectrum(  
    vdos,  
    projection="per_atom",  
    atom_indices=[12, 18, 31],  
    normalize_for_display=True,  
)
```

Deferred work

VP1 intentionally defers:

- overlays of multiple result objects;
- species-group aggregation;
- tensor off-diagonal and complex phase plots;
- peak detection and peak labeling;
- uncertainty bands;
- explicit density conversion from per-THz to per-cm⁻¹ or per-meV;
- automated phonon, infrared, Raman, or neutron-scattering interpretation;
- file writing and interactive plotting.

References

[1] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering* **9**, 90-95 (2007). DOI: 10.1109/MCSE.2007.55.